

Coding & Solution

Date	30June, 2026
Team ID	
Project Name	OptiCrop – Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the implementation quality of the **OptiCrop – Smart Agricultural Production Optimization Engine**. The project is developed using Machine Learning and Flask to recommend the most suitable crop based on soil nutrients and environmental conditions. The code is modular, well-structured, follows Python coding standards, and fulfills the functional requirements of the system.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/khanafiatasleem/OptiCrop-Smart-Agricultural-Production-Optimization-Engine
Programming Language(s)	Python 3.x, HTML5, CSS3, JavaScript
Framework(s) Used	Flask, Scikit-learn, Pandas, NumPy, Joblib, Bootstrap 5
Key Features Implemented	• Soil and environmental data input through web interface • Data preprocessing and validation • Crop recommendation using Random Forest Classifier • Model loading using Joblib • Flask-based web application • Responsive user interface • Crop images and cultivation details • Real-time crop prediction
Pending / Future Enhancements	• User authentication • Prediction history storage • Weather API integration • Multi-language support • Fertilizer recommendation
Setup / Run Instructions	1. Install dependencies using <code>pip install -r requirements.txt</code>. 2. Ensure <code>model.pkl</code> is present in the project folder. 3. Run <code>python app.py</code>. 4. Open <code>http://127.0.0.1:5000</code> in a web browser. 5. Enter soil parameters and click Predict Crop to view the recommendation.

Code Quality Checklist

S.No	Criteria	Status (Yes/No)
1	Code is modular and organized into functions/modules	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments/documentation where necessary	Yes
4	Error handling is implemented for user inputs	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a GitHub repository	Yes

Additional Notes / Comments

- The application uses a **Random Forest Classifier** trained on the Crop Recommendation dataset.
- The web interface is built using **Flask, HTML5, CSS3, Bootstrap 5, and JavaScript**.
- The deployed application is hosted on **Render** and is accessible online.
- The project provides recommendations for **22 different crop types** based on user-provided soil and environmental parameters.
- The solution is designed to be simple, responsive, and easy to use for agricultural decision support.

CodeQualityChecklist

S.No	Criteria	Status(yes/no)
1	Codeismodularandorganizedintofunctions/classes	Yes
2	Meaningfulvariableand functionnamesareused	Yes
3	Codeincludescomments/documentationwhere necessary	Yes
4	Errorhandlingisimplementedforcriticaloperations	Yes
5	Theapplicationrunswithoutcriticalerrors	Yes
6	Codeiscommittedto aversion control repository	Yes

AdditionalNotes/Comments

TheCreditCardApprovalPredictionprojectfollowsgoodsoftwareengineeringpractices by maintaining a modular architecture, reusable code components, and clear documentation. Machinelearningalgorithmsaretrainedandevaluatedtoselectthebest-performing model, which is integrated into a Flask web application for real-time prediction. Theapplicationisscalableandcanbeenhancedfurtherbyaddingdatabase support, authentication, cloud deployment, and prediction history management